Measured Bit Costs and Device-Dependent Capacity for Weight–KV Allocation on 8 GB Edge Devices

Yibo Zhang, Muhammad Syafiq bin Mohd Pozi, and Osman B Ghazali

***Abstract*—Quantization fits the weights of a language model into an 8 GB device, but the cache must fit too, and the two compete for the same bytes. We replace two defaults with measurements. The *objective* is capacity, not latency: 4-bit weights do not buy time, because dequantization is not fused into the matmul on the stacks we run. The *exchange rate* is a property of the operating point and of the quantizer pair. Between shipped quantizers (official GPTQ for weights, KIVI for the cache) one weight bit costs $9.0\times$ the accuracy of one cache bit on Qwen3-0.6B and $13.3\times$ on 1.7B; tightening the weight side alone shrinks that to $1.3\times$, the number this paper defends, and across a weight-by-cache grid a matched $4{\to}6$ interval puts a weight bit at 9.4–$15.5\times$ a cache bit while above 6 bits both curves saturate. The *budget* is device-dependent: the measured unusable-memory fraction is 0.133 on a discrete-GPU laptop, 0.419 on a unified-memory module and 0.047 on a CPU-only board. Only the cache width differs: on the laptop GPU a 32k context fits a shipped runtime only with a 4-bit cache, and on the module that move costs $15.18 \to 18.99$ perplexity. Where the budget binds, at 4B and 32k, allocating bits per layer from these curves measures -2.2% perplexity in $2{,}725$ MB against $+32.5\%$ for uniform 4-bit in $1{,}817$ MB. Grouped-query attention silently disables every fused attention kernel here ($31\times$ memory, $5.8\times$ prefill), and at 2 bits no cache scheme we tried recovers an 11k-token passkey while at 4 bits all do.**

***Index Terms*—bit allocation, edge inference, KV cache, large language models, memory budget, post-training quantization**

I. Introduction

Deploying a large language model on an 8 GB device is no longer a research curiosity; it is the default target for on-device assistants on IoT and edge hardware, and much of the on-device post-training quantization (PTQ) literature is organised around it.

The cache is not the second-order term: That line spends its bits on the weights and leaves the cache at sixteen bits. At deployed context lengths this is arithmetic rather than a hazard: from the published model cards a KV cache costs 112 KiB per token for Qwen3-0.6B and 1.7B and 144 KiB for 4B and 8B, so at 32k tokens the cache is 3.76–4.83 GB against 0.22–4.09 GB of 4-bit weight bodies, before a 16-bit output head that adds 0.31 GB at 0.6B and 1.24 GB at 8B, and the two are equal at 1,919 tokens for 0.6B, 6,144 for 1.7B, 12,319 for 4B and 21,388 for 8B. On an 8 GB device the cache overtakes the weights long before the context lengths these models are deployed at. The question is therefore joint — given a fixed byte budget, how many bits for the weights and how many for the cache — An allocator follows from these curves: exact and cheap, though on the 0.6B checkpoint it does not beat a greedy rule, and its value appears only where capacity binds.

The literature answers the two halves separately: One line optimizes weight precision at a fixed width: rotations to flatten activations [1], [2], [3], [4], Hessian-aware rounding [5], [6], and mixed-precision schemes that spend more bits on sensitive layers [7], [8], [9], [10], [11]. System-level work co-designs the kernel with the format [12], [13], [14], [15], [16], and low-rank compensation recovers what rounding loses [17], [18], [19]. A second line compresses the cache [20], [21], [22], [23], [24], [25], [26] and takes the weight precision as given. Serving stacks that manage the cache as a first-class resource [27] do not choose weights against cache either. The nearest prior art, [23], does map a joint frontier over cache bit-width, cache granularity and chunked prefill — but its weight side is fixed at 4-bit AWQ, so the split is not among its decision variables, its frontier is indexed by memory consumed rather than by a per-bit accuracy cost measured end to end, and it reports no usable-memory fraction for its hardware, so the frontier cannot be re-targeted to a device with a different ρ.

Thesis: three inputs are assumed, and all three are measurable: The allocation literature assumes the objective (speed), the exchange rate between the two sides (practice spends it on the weights), and the budget (the datasheet number). We measure each one, and each measurement reverses the assumption.

The objective. On our stack 4-bit weights cost time rather than saving it, because dequantization is not fused into the matmul; quantization here purchases capacity, not throughput, so latency cannot be the objective of the allocation.

The exchange rate. With published implementations on both sides — the official GPTQ for weights, KIVI's own quantizer for the cache — one cache bit costs $9.0\times$ less accuracy than one weight bit at 0.6B and $13.3\times$ at 1.7B (11.39% against 1.27% perplexity, 3.20% against 0.24%); on sixteen calibration texts the weight cost rises to 13.0% and the ratio falls to $10.2\times$. The ordering reproduces on WikiText-2, and a passkey experiment with fifty prompts per cell locates where the claim stops: at 4 bits every cache scheme we tried retrieves the key, at 2 bits none of them does.

The budget. The fraction of nominal memory unavailable to the model differs $3.15\times$ between a discrete-GPU laptop ($\rho = 0.133$) and a unified-memory edge module ($\rho = 0.419$), both sold as 8 GB. Because the cache grows with context while the weights do not, that difference changes the *relative* return on a weight bit by $3.7\times$ in our analytic cost model: the same drop from 8-bit to 4-bit weights multiplies attainable context

The authors are with the School of Computing, Universiti Utara Malaysia, Sintok, Kedah, Malaysia. Corresponding author: Yibo Zhang (e-mail: zhang_yibo@ahsgs.uum.edu.my).

This work received no specific grant from any funding agency.

by $1.57\times$ on the laptop and $5.76\times$ on the module, while the absolute marginal context per weight bit is close on both ($\sim$3,080 tokens), and the module's measured ceiling falls short of it (Section VIII). A single published configuration cannot be optimal on both.

The generic allocation literature assumes a budget that is uniformly available; on these devices that assumption is wrong by construction, because the nominal number hides a measured ρ that reaches 0.419, so the same fixed recipe that fits one edge class systematically fails on another. A device-aware allocation — one that measures the fraction of memory the model can actually reach instead of reading the datasheet — is therefore not a refinement of the current practice but a prerequisite for deploying the same model across the fragmented memory budgets an 8 GB edge fleet presents.

What we deliver: From those three inputs an allocator follows, and we state its status plainly: it is a multiple-choice knapsack over measured per-device curves, solved by exact dynamic programming, which is textbook. The contribution is not the optimizer, and no published allocator is released for this setting: MultiAlloc and RDKV release no code, ReadKV is a retrieval scheme that needs a serving stack rather than a bit budget, and kvpareto is described only as a frontier indexed by memory already consumed, which is a different input from the budget we solve against. We compare configurations, not allocators, and say so. It is the inputs the optimizer runs on — a measured ρ, a measured workspace curve, and a per-bit accuracy cost measured on both sides of the same model — the finding that the objective has to be capacity rather than latency, and the resulting policy, which we *execute* on a shipped runtime rather than model. On both GPU devices that policy keeps the weights at 8 bits and compresses the cache. At 4k on the laptop it buys 9.97% in perplexity over the deployed nf4 configuration but costs $2.9\times$ in time per decode step, so the length at which it pays is the one where the baseline stops running: a 4B model at 32k allocates 9,033 MiB with an fp16 cache and never completes, against 6,873 MiB and 52.6 tokens/s with the cache packed.

Contributions:

1) **The per-bit cost of the two sides runs against practice, and it is the quantity the allocator needs.** Measured with published implementations on both sides, one cache bit costs $9.0\times$ less accuracy than one weight bit at 0.6B and $13.3\times$ at 1.7B; the ordering reproduces on WikiText-2 ($10.9\times$ and $7.7\times$), so it is a property of the models and quantizers rather than of one corpus, though the magnitude is not corpus-independent. At matched quantizer strength the margin shrinks to $1.3\times$ and $1.6\times$, with 95% intervals that exclude zero but only just, and a full weight-by-cache grid shows the ratio is a property of the operating point: 9.4–$15.5\times$ on a matched $4\to6$ interval, both curves saturating above 6 bits. A passkey experiment with fifty prompts per cell bounds the claim: at 4 bits every scheme we tried recovers the key (50/50), at 2 bits none does (0/50), so the cliff is at the width and not in the grouping (Section VII).
2) **The objective must be capacity, not latency.** On our stack 4-bit weights do not buy time — on 0.6B they cost time outright, because dequantization is not fused into the matmul — so an allocator that minimizes latency optimizes a quantity this hardware does not trade. We give the measured substitution frontier for both models (Section VI-B).
3) **Capacity is device-dependent through a measurable fraction.** The measured unusable-memory fraction is 0.133 on a discrete-GPU laptop and 0.419 on a unified-memory module, a $1.6\times$ difference in budget and a $3.15\times$ difference in the fraction lost; it moves the relative return on a weight bit by $3.7\times$. We report ρ as a measurement procedure rather than a constant, and report the module's *measured* ceiling beside the cost model's prediction for it (Section III).
4) **The policy is executed, not modelled.** We solve the allocation exactly and run the configuration it recommends on a shipped runtime that stores the cache packed. On an 8 GB laptop the recommendation is the difference between a length existing and not (Table VIII).
5) **The stack sets the budget before any bit does.** A silent attention-dispatch failure costs $31\times$ in attention peak memory and $5.8\times$ in 4k prefill time, and removing it extends the attainable context $4\times$ before any quantization decision is made. We release a model-free reproduction, because the effect leaves no trace in the framework's output (Section VI-A).

Scope: The deployment class is the 8 GB IoT and edge machine, a single board or a laptop GPU that must hold weights, cache and runtime in one address space. We hold that class fixed on purpose: it is the setting an IoT deployment actually ships into, where the model and the cache compete for the same bytes and there is no host memory to fall back on. The accuracy measurements are made on two models (0.6B, 1.7B) whose quantizer families we could match on both sides, on two GPU devices and one CPU device, with the power mode fixed; a 4B model carries the system measurements; we did not instrument clocks, so a throttling explanation for a small difference cannot be excluded from our data alone. We do not propose a new quantizer, and we do not claim the ordering is model-independent — the margin narrows as the model grows, which is what the mechanism predicts. We also report a hypothesis we tested and could not support: that the optimal allocation reverses between prefill and decode (the Supplementary Material).

II. Related Work

A. Quantization for speed

The dominant framing in post-training quantization is that fewer bits mean faster inference. Rotations flatten activations so a single 4-bit quantizer suffices [1], [2], [3], [28], [29], [30]; second-order methods cancel rounding error [5], [31]; system papers report speedups from weight-only formats [32]. That framing is what our Section VI-B measures directly, and on our stack it does not hold: the arithmetic stays fp16, so the only quantity that changes is bytes stored.

B. Capacity-oriented compression

A second line treats memory, not time, as the scarce resource: KV cache quantization [20], [21], [22], joint memory–accuracy frontiers [23], and precision allocation under a storage budget [7], [8], [9]. Closest to our setting, [10] scores precision choices separately for prefill and decode and [33] specialises formats per phase, while [16] keeps decode in high precision.

Reference [23] deserves a precise comparison because it is the nearest prior art. It maps a joint frontier over cache bit-width, cache granularity, chunked prefill and weight quantization, and reports Pareto-optimal configurations at 68–78% memory reduction. Its weight side is fixed at 4-bit AWQ, however, so the weight/cache *split* is not among its decision variables; its frontier is indexed by memory consumed rather than by a per-bit accuracy cost measured end to end; and it reports no usable-memory fraction for its hardware, so the frontier cannot be re-targeted to a device with a different ρ. Our claim is narrower and different: that the per-bit cost of the two sides must be measured on the same footing, that the resulting optimum is device-dependent through ρ, and that on our stack the answer is to compress the cache and leave the weights alone.

Two recent systems are adjacent enough to name rather than let the reader guess. [34] allocates precision *inside* the cache under a fixed KV memory budget, layer-wise and token-wise, so it is the closest in spirit to our budget treatment; but its decision variables are the KV precisions themselves, not the weight/cache split, it is constrained by a KV budget alone rather than by a measured device capacity, and it does not model the dequantization throughput cost of lowering a bit. [35] is an algorithm–hardware co-design that hybridises online/offline KV quantization with a dedicated quantize/dequantize engine; it is a throughput optimization for serving, not a capacity- or accuracy-budget framework, and it likewise reports no usable-memory fraction.

Two lines of work focus the same question on the memory-constrained devices this journal serves — mobile, embedded and edge modules — rather than on servers, and both are closest in spirit to our budget-first view. [36] is an overhead-aware KV cache loading framework for on-device LLM inference: it models the cost of each KV chunk and decides, per chunk, whether to stream it from the cloud or compute it locally, overlaying the two paths and re-tuning the schedule at runtime as wireless and device conditions change. [37] learns an adaptive policy that assigns each token a cache precision from {2,4,8,FP16}-bit proportional to its importance, improving the accuracy–latency trade-off on sub-2B SmolLM models relative to static KV quantization. Neither work, however, makes the weight/KV precision split a decision variable under a *measured* device budget: both fix the weight side at a published precision and allocate only within the cache, and neither reports the device's usable-memory fraction. Our allocator sits *above* both — it decides which side of the split the scarce bytes go to, given a measured ρ — so the adaptive-cache precisions of [37] and the load/no-load decisions of [36] are the layers it budgets for, not alternatives to it.

[38] inverts the usual motivation and deserves a candid mention: on a 4 GB Turing GPU it finds that under sustained decode a 4-bit cache decodes *slower* than fp16, by 12–50% depending on depth, because the dequantization kernel adds work to every token. That is precisely the throughput-side hedge a capacity-saving claim must survive, and it is why our allocator scores bits against both the accuracy cost we measure and the dequantization throughput of the shipping kernel rather than assuming a lower bit is free.

Three further lines sit closest to ours. RDKV casts cache compression as a rate-distortion problem and allocates bits across tokens and channels between eviction and quantization [39]; ReadKV separates retained bits from bits fetched per query, beating query-independent readers at the same read budget [40]; Q-Palette makes the bit-width itself the decision variable, on the weight side, with fractional formats [41]. RDKV and ReadKV are the closest prior art to our cache side, and each already chooses bits rather than a fixed width. What none treats is the *split* between weight bits and cache bits under a measured device budget, or the usable-memory fraction of the target device, which dominates our results once capacity binds.

C. Edge measurement and reporting

A recurring criticism is that quantization results are not reproducible because the execution stack is unreported. Our measurements give that criticism a number: the same model on the same card supports 4k or 16,384 tokens of context depending only on which attention kernel the framework selects (Section VI-A). This is why we record the software stack and the device's memory state with every latency number, and why ρ is reported as a procedure rather than a constant.

III. Usable Memory and the Weight-Bit Exchange Rate

What "8 GB" actually means: We measure the fraction of nominal memory that is unavailable to the model on the day of the experiment, rather than trusting the datasheet. On the laptop this is the CUDA context plus the compositor, read from the allocator; on the Jetson it is L4T, the desktop and the carve-out, read from the kernel. The difference is large enough to change conclusions: 0.133 leaves 6.93 GiB usable on the laptop and 0.419 leaves 4.32 GiB on the module, so the usable budgets differ by $1.6\times$; all three are sold as 8 GB, and the CPU-only board, where the OS is the only claimant, sits at 0.047. ρ is not a constant: it moves with the OS image, with background load, and — on the Jetson — with the power mode, which we set once (25 W) and did not vary, so power-mode sensitivity is a stated caveat rather than a measurement. A device and a bit-width quoted without ρ do not transfer.

The exchange rate between a weight bit and a token: Because the cache grows linearly in context and the weights do not, a saved weight bit converts into context at a rate that depends on how tight the budget is. For Qwen3-4B with the cache held at 16 bits, dropping weights from 8 to 4 bits multiplies the attainable context by $1.57\times$ on the laptop and by $5.76\times$ on the module, so the same bit buys $3.7\times$ more

relative context on the constrained device. Both figures are model outputs. The module is not plotted: its measured ceiling is 3,072 tokens, so a curve that reaches tens of thousands there would describe a configuration that cannot run.

Why this is a capacity argument, not a speed argument: It would be natural to read the numbers above as "quantize the weights to go faster". They do not say that: Section VI-B shows the 4-bit path is slower on this stack. What they say is that on a device this constrained, precision decides how much context can be addressed at all, and that the exchange rate for that knob is device-dependent.

IV. Method

Why the objective is capacity, not latency: The natural formulation is to minimize serving time subject to an accuracy constraint. We tried that first and it is degenerate on this stack: latency is not a function of the bit-widths at all, since a 4-bit weight is still multiplied at fp16 precision (Table V). An objective whose gradient with respect to the decision variables is zero cannot rank allocations. What bit-width *does* determine here is how much context fits, so that is what we optimize: maximize the attainable context subject to a bounded accuracy loss. This is a measured choice, not a modeling convenience, and it is the reason Table V appears before the method rather than after it.

Budget constraint: Let M be nominal memory and ρ the measured unusable fraction. An allocation $\mathbf{b}$ is feasible for a context length L iff

$$\sum_{l} \left(b_l^W \frac{|W_l|}{8} + s_l \right) + 2 n_\ell L d H_{kv} \frac{\bar{b}^{KV}}{8} \;\le\; M(1-\rho) - W_{\text{act}}(L), \tag{1}$$

where s_l is the scale overhead of group l, n_ℓ the number of layers, H_{kv} the key/value heads and d the head dimension; the leading 2 counts the key and value tensors, and $\bar{b}^{KV}$ is a nominal per-element budget that already contains the per-group scales, so no separate cache-scale line appears; the cache term is linear in L. The workspace term is the one prior budget model gets wrong: it is neither zero nor constant. Measured, it is linear in the tokens of a single forward pass with a *model-dependent* coefficient and intercept: 0.142 MB per token and 9 MB for the 0.6B model, 0.217 and 161 MB for the 4B. The coefficient must be refitted per model: the 4B value applied to 0.6B over-reserves workspace by about 60%, a mistake we made and corrected. What it does not depend on is the device: the 0.6B curve fitted on the laptop predicts the Jetson's measured workspace to within 2% (Section V). Because the term scales with the tokens in a single forward pass rather than with the conversation, chunked prefill bounds it at $W_{\text{act}}(L) = c \cdot \min(L, C) + c_0$ with chunk size C. Fitting the whole prompt in one pass is what makes 32k look infeasible on an 8 GB card; it is a property of the serving stack, not of the device.

Objective: For a target context length L,

$$\min_{\mathbf{b}} \sum_i \Delta\text{PPL}_i(b_i) \quad \text{s.t.} \quad (1), \tag{2}$$

where the sum runs over every weight matrix and every KV group and $\Delta\text{PPL}_i(b)$ is the perplexity cost of quantizing unit i to b bits, measured per unit (Section V). The capacity question *at an accuracy budget* ϵ is the inverse: the largest L for which (2) attains a value below ϵ. We report both directions, since the second is what a deployment engineer asks and the first is what shows *where* the bits went.

1) **Inputs** — for each unit i (weight matrix or KV group), the measured cost $\Delta\text{PPL}_i(b)$ at $b \in \{2,3,4,8\}$, and the byte cost $\frac{b}{8}|W_i| + s_i$; the measured budget $B = M(1-\rho) - W_{\text{act}}(L)$ from (1).
2) **Discretise** — round every option's byte cost *up* to a multiple of a bucket; the grid must be finer than the number of units.
3) **Solve** — multiple-choice knapsack over the buckets: for each bucket, keep the least ΔPPL assignment that reaches it; the exact optimum and the full capacity–accuracy Pareto frontier come out in one pass in $O(n \cdot |\mathcal{B}| \cdot \text{buckets})$.
4) **Report** — the argmin allocation $\mathbf{b}^*$, and the inverse-capacity question: the largest L whose (2) value stays below ϵ.

Fig. 1. The allocator as a finite sequence.

Solver: With $\mathbf{b}$ discrete per unit, (2) is a multiple-choice knapsack and dynamic programming over discretised byte buckets returns the exact optimum in $O(n \cdot |\mathcal{B}| \cdot \text{buckets})$; no heuristic search is needed, and the full Pareto front comes out in one pass. Two implementation details matter for correctness and cost us two bugs to learn: bucket costs must be rounded *up* (rounding down lets the solver quietly exceed the budget), and the bucket grid must be finer than the number of decision units (otherwise every unit's minimum cost of one bucket makes a feasible problem look infeasible). Figure 1 lays the allocator out as a finite sequence: a per-unit cost table, a budget that is closed over measured inputs rather than assumed, and an exact knapsack pass that returns the whole capacity–accuracy frontier in one shot.

Do simpler rules suffice?: On the 0.6B item table the budget never binds up to 32k: the exact optimum, uniform allocation and a greedy upgrade rule all return the widest width that fits, within 0.03 of each other in summed proxy error, and the greedy rule actually spends fewer bytes than the exact solver (738 against 810 MB at 4k). The shipped 4-bit-plus-16-bit-cache recipe costs 0.36 in accuracy for 7% fewer bytes at 4k and 63% *more* bytes at 32k. Allocation has work to do only where capacity binds, and there it does work. On 4B at 32k the budget is 2,747 MB, against 1,817 MB for uniform 4-bit weights and 3,633 MB for 8-bit. Solving the same exact knapsack over a per-layer curve (36 layers, measured at 4,096 tokens) and restricting it to 4 and 8 bits gives -2.23% perplexity in 2,725 MB, where uniform 4-bit costs $+32.49\%$ in 1,817 MB: a 35-point gain, which the additive model predicted to within 8 points. Admitting 3 bits does not pay — it measures $+8.23\%$ — because the per-layer cost is superadditive where a layer is badly quantised.

What is measured and what is assumed: ρ, $W_{\text{act}}(L)$, B_{eff} and P_{eff} are measured per device. The per-unit costs ΔPPL_i are measured one layer group at a time at 2, 3, 4 and 8 bits, and for the cache globally. We tried per-linear resolution first and abandoned it: across 252 linears the average cost at 4 bits is 0.11%, below the 0.42% floor of a single measurement, so those numbers are noise rather than signal. The layer curves are used as measured, with no rescaling; the additive model built on them over-predicts the end-to-end gain by about eight points, as the allocation test below shows. Spreading the cache

TABLE I
THE THREE DEVICES ON ONE PAGE. "USABLE" IS WHAT THE MODEL CAN ADDRESS AFTER THE OS AND DRIVER RESERVE THEIR SHARE; THE CEILING IS THE LONGEST CONTEXT WE MEASURED, WITH THE CACHE WIDTH THAT ACHIEVES IT.

Device	nominal	usable	ρ	ceiling
A, 4060 laptop	8,188 MiB	6.93 GiB	0.133	32k, spills
B, Orin Nano Super	7,619 MiB	4.32 GiB	0.419	3,072 tok
C, Pi 5 (CPU)	8,048 MB	7,809 MB	0.047	64k, 4-bit

TABLE II
QWEN3-1.7B PERPLEXITY ON DEVICE C, SAME 2,000-ABSTRACT CORPUS, 512 TOKENS AND 20 CHUNKS, WITH LLAMA.CPP K-QUANT WEIGHTS. PARENTHESES GIVE THE CHANGE AGAINST THE `Q8_0`, 16-BIT-CACHE CELL.

weights	16-bit cache	8-bit cache	4-bit cache
`Q8_0`	34.214	34.248 (+0.10%)	41.216 (+20.46%)
`Q4_K_M`	34.159 (−0.16%)	34.479 (+0.77%)	41.910 (+22.49%)

cost uniformly across layers is an assumption, stated as one, because a per-layer cache measurement was not available.

V. EXPERIMENTAL SETUP

Devices: Three nominally 8 GB targets, chosen because they span the memory architectures an edge deployment uses: discrete, unified, and CPU-only. *Device A* is an RTX 4060 Laptop GPU (8 188 MiB, sm_89, driver 592.82, measured 161.9 GB/s copy bandwidth and 22.36 TFLOP/s FP16 at batch 128). *Device B* is a Jetson Orin Nano Super (7 619 MiB unified LPDDR5, L4T R36.4.7, sm_87, 8 SMs, measured 54.1 GB/s copy bandwidth and 4.51 TFLOP/s FP16 at batch 128). We report its *usable memory* as measured — 7 619 MiB unified against 4.32 GiB actually usable (ρ=0.419; the laptop is 8 188 MiB against 6.93 GiB, ρ=0.133) — and we state plainly which of its numbers are measured and which are derived, because the distinction matters. *Device C* is a Raspberry Pi 5 (8 GB LPDDR4X, four Cortex-A76, CPU only, llama.cpp), where no GPU reserves memory: 8,048 MB installed, 7,809 MB available after boot (ρ=0.047). It exists to show which effects outlive the GPU. Its budget, ceiling and accuracy grid are reported alongside the other two in Tables I and II; the energy measurement of Section VI-E does not extend to it, because the board exposes no per-rail telemetry. Cache bits, not weight bits, set the cost here too: at B=1, 2, 4 a 4-bit cache costs 21% of prefill at 512-token prompts (53.6 to 42.1 tok/s), rising to 58% at 4,096 (39.7 to 16.5), and it leaves decode untouched (6.28 against 5.99 at B=1). The same three weight files, same corpus and same command line on all three devices give 34.40, 34.29 and 42.50 on Device A, 34.34, 34.28 and 42.49 on Device B, and 34.21, 34.16 and 42.40 on Device C: Q4_K_M sits 0.16 to 0.31 per cent below Q8_0 and Q2_K 23.6 to 23.9 per cent above it, so the cost of a weight bit is a property of the model and the quantiser, not of the device that runs it.

- **Measured on Device B**: total and available memory, hence ρ; copy bandwidth (54.1 GB/s) and achieved FP16 throughput (4.51 TFLOP/s at batch 128), hence b^*_{Jetson}=83; and an end-to-end sweep of the real 0.6B checkpoint at 512–2,048 tokens (TTFT 165.1, 307.6 and 638.0 ms; ∼108 ms per decode step; peaks of 1,334, 1,468 and 1,734 MB).
- **Derived, not measured, on Device B**: the context lengths in its workspace peaks quoted for it in Section VI-A. Those follow from the measured ρ together with a workspace curve fitted on Device A. That transfer is now evidence-backed rather than assumed: the fitted curve $0.142L + 9$ MB predicts the three measured Jetson peaks to within 2% (83, 158 and 307 MB against predictions of 82, 154 and 300), so the workspace is a property of the model and the framework rather than of the GPU. The *lengths* remain analytic because the naive attention path exceeds the module's usable memory above 2k tokens, which is itself the Section VI-A effect.

Software stack: Every number carries the stack it was measured on, because Section VI-A shows that the stack can move attainable context by 4×. Unless stated otherwise: torch 2.6.0+cu124, transformers 5.20.2, bitsandbytes 0.50.2, Windows 11 on Device A, L4T on Device B and Debian 12 on Device C. Weights come from ModelScope; the network in our environment cannot reach HuggingFace, and §V of the artifact records the package indices used.

Measurement protocol: Each configuration is measured by (i) loading and warming up, (ii) timing three prefill passes over the full prompt and taking the median, (iii) timing eight decode steps against a pre-built cache and taking the median. Timings are taken under `torch.inference_mode`; prefill is timed on the base model so that the language-model head is not applied to every position (Section VII). Peak memory is read from the CUDA allocator after each phase separately, and any run whose peak exceeds 0.92 of device memory is flagged as spilled and excluded rather than reported: on Windows the driver backs an over-budget allocation with host memory instead of raising an error, which degrades throughput by up to 30× silently.

Interleaving: Configurations that are compared are measured *interleaved*, not one after the other. Background load moved a short-context prefill time by 10% between two consecutive runs of the same configuration in our own measurements; an earlier draft compared nf4 and fp16 measured minutes apart and reported the wrong ratio. All ratios in Table V are from interleaved runs.

Models: Qwen3-0.6B, Qwen3-1.7B and Qwen3-4B, with configurations taken from the published model cards, whose per-token cache sizes are 112 KiB (0.6B, 1.7B) and 144 KiB (4B, 8B). The 0.6B and 1.7B models carry the accuracy measurements, where a full-precision reference still fits; the 4B model carries the system measurements, where one does not. Sizes above 4B are analyzed but not run: their 4-bit weights plus a 16-bit cache exceed 8 GB before any context is added, which is itself the motivation for the allocation problem.

What is synthetic, and why it is legitimate: Once the real 0.6B checkpoint arrived we found that an architecture-exact random-weight model reproduces its memory figures within 1% and its naive prefill time within 4%, because latency and footprint depend on tensor shapes and dtypes rather than on

TABLE III
ATTENTION PEAK MEMORY, q=16 HEADS, kv=8 HEADS, L=4096, d=128, FP16. IDENTICAL OUTPUT VALUES IN EVERY SUCCESSFUL ROW.

KV heads	backend	peak
as-is	memory-efficient	*no kernel*
as-is	flash	*no kernel*
as-is, `enable_gqa`	math	2,504 MB
materialised	memory-efficient	**80 MB**
materialised	math	**2,693 MB**

weight values. All numbers in Tables IV and V are from real weights; synthetic values are kept in the artifact as a cross-check.

Accuracy measurement: Perplexity is measured on a 575k-token corpus assembled from 2,000 arXiv cs.CL/cs.LG abstracts, and layer-wise sensitivity by quantizing one layer group at a time at 2, 3, 4 and 8 bits (144 measurements on 36 layers of the 4B checkpoint). The corpus is a documented fallback: WikiText-2 was not reachable during the main experiments, so absolute perplexities are not comparable to the literature and only relative increases are used. A single measurement at this granularity has a floor of about 0.4 points, which is why the cost table is kept at layer rather than per-linear resolution.

VI. RESULTS

Every system number below is measured on the target hardware with a real checkpoint. Table VI is the exception and is analytic by construction: it follows from the measured inputs named in Section V, and its workspace model is validated on Devices A and B. Accuracy numbers use the abstract-level corpus and are reported as relative perplexity only.

A. The stack decides the budget before any bit does

On this stack — RTX 4060 Laptop, Windows, torch 2.6.0+cu124, transformers 5.20.2 — a Qwen3-style model never reaches a fused attention kernel. Its 16 query heads and 8 KV heads violate the equal-head requirement of the flash and memory-efficient backends, so PyTorch dispatches to the math backend, which materialises the score matrix in fp32. The failure is silent: no exception, no warning that survives into a log.

Table III isolates the cause. With identical tensors and identical mathematics, materialising the KV heads makes the fused kernel available and reduces the attention peak by 31×; we verified numerically that the two paths agree.

`enable_gqa` is the route a caller would try first, and it does not help: the flag is accepted, but the call still materialises the score matrix — 2,504 MB and 165 ms at L=4096, against 80 MB and 24 ms once the heads are expanded. The only route to a fused kernel here is to expand the KV heads in the caller, which the SDPA path in transformers 5.20.2 does not do.

Table IV shows what this does to end-to-end serving. Restoring the fused path moves the attainable context from 4k (with a silent spill at 8k) to 16,384 tokens, and cuts prefill time at 4k by 5.8×. At 8k the naive path's 71 s is a wall-clock figure for a spilled run and we do not treat it as a latency measurement; the fused path completes the same prefill in 0.99 s, a 72× wall-clock difference that a deployment would feel even though it is not a clean latency ratio. At the 4B scale the effect is qualitative rather than quantitative: with 4-bit weights and the fallback, a 4k prompt already exceeds the card (9,195 MB peak, measured with the real checkpoint) and the run degrades to 47 s per prefill, whereas the fused path completes the same prefill in 1.9 s with 2.7 GB to spare. That is not a slow configuration but an unusable one, and nothing in the framework reports it.

TABLE IV
END-TO-END EFFECT OF THE ATTENTION DISPATCH, ON REAL CHECKPOINTS AT BOTH SIZES. WE FIRST SCREENED THE 4B SCALE WITH AN ARCHITECTURE-EXACT RANDOM-WEIGHT MODEL AND THEN RE-RAN EVERY ENTRY WITH THE REAL CHECKPOINT: THE SYNTHETIC MODEL PREDICTED PEAK MEMORY TO WITHIN 0.7% AND PREFILL TIME TO WITHIN 8%, SO THE SCREENING PROCEDURE IS SOUND, BUT EVERY NUMBER BELOW IS REAL-WEIGHT. "SPILLED" MARKS RUNS THAT EXCEEDED VRAM AND WERE BACKED BY SYSTEM MEMORY WITHOUT RAISING AN ERROR, SO THEIR TIMINGS ARE NOT USABLE AS LATENCY.

L	TTFT naive	TTFT fused	peak naive
	Qwen3-0.6B, fp16		
1,024	164.7 ms	**62.2 ms**	1,644 MB
4,096	2,062.5 ms	**355.9 ms**	4,836 MB
8,192	71,420 ms (spilled)	**985.0 ms**	13,436 MB
16,384	out of memory	**2,795.2 ms**	—
	Qwen3-4B, nf4		
1,024	692.3 ms	**425.3 ms**	3,433 MB
4,096	*spilled* (46.7 s)	**1,892 ms**	9,195 MB
8,192	—	**4,356 ms**	5,795 MB

The fix is not free everywhere, and the second device shows the boundary. Repeating the sweep on the Jetson gives 165.1, 307.6 and 638.0 ms of TTFT for the naive path at 512, 1,024 and 2,048 tokens, against 189.4, 354.6 and 833.3 ms with the KV heads materialised — the rewrite is 15–31% *slower* there. At these lengths the $O(L^2)$ term is too small to pay for the Python-level repeat, and the module's CPU is slow enough that the overhead shows. On the laptop the crossover falls between 2k and 4k, which is also where the naive path starts to spill; on the Jetson we cannot reach 4k at all, because the framework materialises logits for every position (1.2 GB at L=4096 for a 152k vocabulary) on top of a model and cache that already occupy half the usable memory. Applying the dispatch fix below roughly 2k tokens is therefore a pessimisation, and it should be gated on context length rather than enabled globally — a caveat we did not anticipate when we first measured the 31× memory reduction.

B. Quantization buys capacity, not speed

Table V compares 4-bit nf4 weights against fp16 on the real checkpoint with the same attention path, measured in an interleaved fashion so that both configurations see the same background load. nf4 is *slower* at every length up to 8,192 tokens, by 21% at 1k and 7% at 16k, and ties at 16,384. The mechanism is that nf4 stores 4-bit weights but performs an fp16 tensor-core matmul, so the FLOPs are identical, while

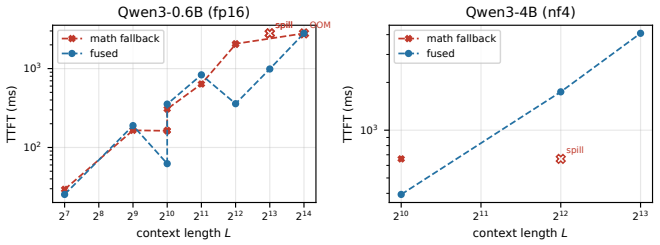


Fig. 2. Time to first token versus context length. Crosses mark runs that spilled or ran out of memory; they are drawn rather than dropped, because where the naive path stops working is the point.

TABLE V
4-BIT NF4 VERSUS FP16 ON THE REAL QWEN3-0.6B CHECKPOINT, FUSED ATTENTION PATH. EVERY ROW IS THE MEAN OF TWO INTERLEAVED RUNS, SO BOTH CONFIGURATIONS SAW THE SAME BACKGROUND LOAD. NF4 IS SLOWER AT EVERY LENGTH UP TO 8,192 TOKENS AND TIES AT 16,384.

	TTFT		TPOT	
L	fp16	nf4	fp16	nf4
1,024	65.4 ms	79.4 ms (1.21×)	26.9 ms	37.1 ms
4,096	356.9	404.2 (1.13×)	28.8	37.2
8,192	974.5	1,063.4 (1.09×)	50.7	71.5
16,384	2,765.0	2,945.1 (1.07×)	75.5	75.4

the unfused dequantization adds elementwise traffic that the saved bytes do not pay for.

A third precision makes the point sharper still. Bit-width does not even *order* latency on this stack: bnb's 8-bit path stores 752 MB against nf4's 531 MB, yet costs 520.9 ms of TTFT at 4k against nf4's 404.2 ms and fp16's 355.9 ms, and 129.2 ms per decode step against 37.2 and 28.8. The reason is that bnb's int8 path performs per-multiply outlier decomposition, which is a property of that kernel and not of 8-bit arithmetic.

Nor is the ordering stable across phases or model sizes. At 4B the same comparison reverses for prefill and widens for decode: int8 (4.41 GB of weights) prefills 4k in 1,613 ms against nf4's 1,744 ms, while decoding at 128.5 ms per token against 54.7. A single "quantization speedup" number is therefore not meaningful without naming the phase and the model size.

The consequence for method design is direct: an allocation objective written in terms of latency is degenerate on this stack, because latency does not depend on the decision variables. We therefore optimize capacity subject to an accuracy budget.

Batch $>$ 1: the throughput cost, and what this sweep cannot test: Serving is not single-stream, so we repeat the comparison with a batch of B sequences sharing the GPU. Per-sequence decode throughput for the simulated-cache configuration collapses as the batch grows, because the unfused dequantization re-runs per token, per sequence, in the cache's place. On Qwen3-0.6B at L=1,024 it falls 9.97 → 9.48 → 2.34 tok/s as B goes 1 → 4 → 16, against 13.16 → 12.99 → 11.64 for the nf4 baseline whose aggregate throughput scales linearly, and the same collapse appears at the other end of the edge scale: on SmolLM2-135M the simulated-cache configuration falls from 25.4 to 2.7 tok/s between B=1 and B=16, against 31.8 for nf4 at B=1. **This sweep therefore measures what cache compression costs in throughput; it cannot measure what it buys in capacity.** The in-attention quantizer we use here quantizes and immediately dequantizes, so it leaves the allocated KV footprint unchanged (114,688 B per token in both configurations), and the fp16 weight copy makes the simulated-cache row larger at every cell (3,569.5 against 2,580.7 MB at B=16). At B=16, L=4,096 both configurations spill (8,620.9 and 10,018.6 MB), so no cell in this sweep shows a batch the baseline cannot run and the compressed cache can; only the packed-cache runs of Section VI-D show that, and they are single-stream.

Two boundaries bound this claim. At 4 B the policy configuration cannot be measured on either device: its full-precision weights alone need 8.0 GB, over the budget, so the whole policy column spills or aborts. A 4-bit k-quant 4 B does run on the module at 2,048 tokens — 18.5 tok/s with an fp16 cache and 18.1 with a 4-bit one — but the nf4 baseline of Table III itself spills at L=4,096,B=16 (peak 8.9 GB) and OOMs at B=16, L=4,096. On the Jetson the small cells reproduce the same ordering (naive16 8.71 vs. policy 6.0 tok/s at B=1,L=1,024; 7.07 vs. 2.15 at B=4), but the in-kernel fake-quantization path does not shrink the KV footprint on that stack, so those policy cells measure quantization overhead on an uncompressed cache and are reported as a boundary rather than a result; the unified-memory allocator also aborts outright at L=4,096 with $B \geq 4$.

Energy per token does not follow the cache: On the module, board power (`VDD_IN`) rises from 14.5 W at 4k with an fp16 cache to 16.1 W at 32k with a 4-bit cache, while decode throughput stays near 24 tokens/s: energy per token moves from 595 to 664 mJ (one run each), and an 8-bit cache at 16k sits between at 648 mJ. Length and cache width move together across these three cells, so they do not separate the two effects, and the marginal cost actually falls at 32k (326, 250 and 310 mJ after subtracting idle). We report totals only: on these cells the weight stream dominates what a token costs.

C. Capacity is device-dependent

Table VI solves the allocation problem on the laptop for a range of accuracy budgets. The module is not solved the same way because the model does not hold there; its usable budget is simply smaller, by the factor Section III measured.

Every row of Table VI lies above the 40,960-token window Qwen3-0.6B was trained on, so none of them is a context a user can reach: with a C = 2,048 prefill the cache does not bind on this checkpoint at all, and the arithmetic only says the budget would allow more. The binding cases in this paper are measured, not extrapolated: a 4B model at 32k on Device A and a 1.7B model at 64k on Device C, where the fp16 cache fails to allocate and a 4-bit cache runs (Table VII).

The module ceiling is measured, and it is where the model breaks: The workspace fit comes from the laptop and has no logits term: the framework materialises logits for every prompt position, about 1.2 GB at 4,096 tokens, which is why the single-pass model overestimates the module by more than 20×. We did not add a chunked-prefill term, so no module

TABLE VI
LARGEST QWEN3-0.6B CONTEXT ON DEVICE A WHOSE PERPLEXITY INCREASE STAYS WITHIN ϵ, FROM THE MEASURED ρ, WORKSPACE AND WEIGHT/CACHE CURVES. IT IS MODEL-PREDICTED FOR THE LAPTOP ONLY, UNDER A CHUNKED PREFILL OF $C = 2{,}048$ TOKENS, WHICH IS WHAT BOUNDS THE WORKSPACE TERM IN EQ. (1); THE MODULE IS NOT EXTRAPOLATED BECAUSE ITS MEASURED CEILING IS 3,072 TOKENS AND A 4,096 PREFILL ABORTS (SECTION V). WEIGHTS STAY AT 8 BITS WHEREVER THE BUDGET ALLOWS (7.98 WHERE ONE GROUP IS TRIMMED); THE BUDGET GOES TO THE CACHE.

	max context	bits chosen	
ϵ	4060	weights	cache (0.6B / 1.7B)
0.01	111,184	8.00	7.7 / 7.4
0.02	122,404	8.00	6.9 / 6.6
0.05	176,464	8.00	4.8 / 4.6
0.10	221,344	7.98	3.8 / 4.1
0.20	243,784	7.98	3.3 / 3.4

column is reported. Measured directly, increasing prefills 1,024 tokens run at a peak of 1.52 GB, 2,048 at 1.92 GB and 3,072 at 2.32 GB, while 4,096 aborts rather than degrading. The failure is not a clean OOM exception — it is an NVML assertion inside the caching allocator, after the driver reports `NvMapMemAllocInternalTagged error 12` — which is why the bound can only be located by running each length in its own process.

We say so rather than let the number stand. The model behind Table VI contains the measured ρ and a workspace curve fitted to that device, and it reproduces the 512–2,048 range to within 2%. What it does not capture is that this device fails *in blocks*: the allocator aborts on a large allocation instead of paging or fragmenting its way through, so the smooth extrapolation is simply not the operative mechanism past a few thousand tokens. The laptop, which spills instead of aborting, does run at 32k (Table VII), which is the same failure mode appearing benign on one device and fatal on the other.

What is *flat* at 1.63–1.82× across a twentyfold range of budgets is the model's between-device *context* ratio. The laptop column exceeds the trained window of 40,960 tokens at every budget, so those cells are arithmetic rather than attainable context; the ratios this paper claims are the *budget* ratio, 1.60× (6.93 against 4.32 GiB), and the *fraction* ratio, 3.15×. The weight column is pinned at 8 bits everywhere. The solver is free to choose any per-layer width and declines to spend on weights — not because we forbade it, but because the measured curves make the cache the better buy at every budget shown (Table X).

What the default actually costs, measured end to end: Two natural ways to make this comparison are both wrong, and the errors are instructive. Pricing the weight side from a cost curve at our own 4-bit round-to-nearest makes the default look catastrophic (36.5%); real `nf4` is a far better quantizer than plain rounding, so that is a strawman. Comparing `nf4` weights against full-precision weights that carry a *simulated* 4-bit cache makes the default look three times faster; that measurement includes a software dequantization step on every layer and therefore describes our patch rather than the default.

Table VII carries the control that settles it: full-precision weights with an untouched cache, so the only difference from

TABLE VII
FOUR CONFIGURATIONS DEPLOYED AT L=4096 ON DEVICE A OVER FOUR WINDOWS OF 1,024 TOKENS. ΔPPL IS RELATIVE PERPLEXITY AGAINST THE `NF4` ROW, SO NEGATIVE IS BETTER. "FULL" IS FULL-PRECISION WEIGHTS WITH AN UNTOUCHED CACHE: THE CLEAN CONTROL FOR THE WEIGHT-PRECISION EFFECT, SINCE NO CACHE QUANTIZATION SITS ON ITS PATH. THE TWO CACHE-COMPRESSED ROWS ARE *simulated* — OUR ATTENTION PATCH QUANTIZES AND IMMEDIATELY DEQUANTIZES — SO THEIR LATENCY AND MEMORY ARE UPPER BOUNDS RATHER THAN WHAT A PACKED 4-BIT IMPLEMENTATION WOULD REACH; THEIR PERPLEXITY USES THE SAME IN-ATTENTION QUANTIZER, WHOSE COST DEPENDS ON THE WEIGHT WIDTH: THE TWO 1.7B ROWS COINCIDE WITHIN 0.01 POINTS AT 16-BIT WEIGHTS, WHILE TABLE VIII CHARGES THE SAME CACHE 1.98% WHERE THE WEIGHTS ARE ALSO 4-BIT. LABELS GIVE WEIGHT/CACHE PRECISION; THE 8-BIT CASE IS TABLE VIII.

	config	ΔPPL (%)	MB	TTFT	TPOT
0.6B	nf4	0.00	2,166	450	32.2
	fp16/C4	**−9.97**	2,789	465	92.9
	nf4/C4	+2.22	2,167	516	93.2
	full	**−10.84**	2,810	**402**	**27.0**
1.7B	nf4	0.00	2,949	881	**32.5**
	fp16/C4	**−4.68**	4,942	859	100.8
	nf4/C4	+0.08	2,948	954	96.1
	full	**−4.68**	4,998	**794**	35.3

the `nf4` row is the weight precision. Three things follow.

First, on 0.6B at 4k the published practice is strictly dominated. It is 10.84% worse in perplexity, 12% slower to first token, 19% slower per decode step, and saves 644 MB — which the device does not need, since both configurations fit. There is no dimension in which it wins.

Second, on 1.7B it becomes a real trade, because 3.4 GB of fp16 weights is worth compressing: `nf4` saves 2 GB and decodes 9% faster at a cost of 4.68% perplexity. That is a defensible engineering choice at 4k, and we do not claim otherwise.

Third, compressing both sides is not the answer either: the row labeled *nf4/C4* lands within 0.1–2.2% of the default while the cache is small enough at this length for the saving to be pointless.

The two cache-compressed rows are simulated: our attention patch quantizes and immediately dequantizes, so it pays the arithmetic cost of compression without storing packed bits. Their latency and memory therefore bound what a packed implementation would cost, not what it would achieve, and no accuracy claim in this paper rests on them.

The *memory* claim, however, is not left modelled. We ran a prefill at 8k and 32k, captured every layer's key and value tensors, packed them to 4 bits two values per byte with a per-32-group scale and minimum, and asked the allocator how much it actually holds. At L=8192 the fp16 cache is 902 MB and the packed form is 282 MB; at L=32768 they are 3,611 MB and 1,129 MB. Both are a 3.2× reduction — below the ideal 4× because a 5-bit budget covers the nibbles plus one bit of scale, which is the arithmetic the capacity table assumes. What this does not demonstrate is a decode step running on the packed form; that needs a chunked dequantizing attention kernel, which we did not write, and the capacity argument needs only the resident footprint.

What makes the default the wrong choice regardless is the length at which it stops fitting. The cache grows with L and the

weights do not, and on this laptop the driver spills to system memory rather than raising an error, so the failure is gradual: at 32k the 0.6B `nf4` configuration already peaks at 13.7 GB against 6.9 GB of dedicated memory, and its decode time per token rises from 32 ms to 1,076 ms as a result. Section VI-D measures the same crossing on a packed runtime rather than estimating it: a 4B model at 32k allocates 9,033 MiB with an fp16 cache and does not complete, and 6,873 MiB with the cache at q8_0 and does. The honest summary is conditional: *at 4k the default is a modest trade on the larger model and a dominated choice on the smaller one; past roughly 16k on a 6.9 GB budget it is not a trade at all, and cache compression is what makes the length reachable*.

Is the policy an artefact of a weak weight quantizer?: That is the obvious objection, and it separates from the ordering question of Section VI-E: even if a cache bit is cheaper, a good enough weight quantizer might still be worth using. We tested it directly by rebuilding the allocator's inputs with the weight side at the *tuned* cost of Table X (6.97% at 4 bits instead of 34.65%, a factor of five) and re-solving. Almost nothing moves: at ϵ=0.01 both GPU devices still choose 8-bit weights and the attainable context is unchanged to the token; at ϵ=0.20 the mean weight width is 7.98 on the laptop and 8.00 on the Jetson with the default curves, against 7.96 and 7.98 with the tuned ones. The reason is geometric rather than numerical: at these context lengths the cache is the dominant consumer of bytes, so a bit moved to the cache buys far more context than the same bit buys in accuracy. The policy is therefore robust to the strongest objection we can raise against it, even though the underlying ordering is not (Section VI-E).

D. The recommended policy, executed

The allocation our procedure recommends is easy to state: weights at 8 bits, cache compressed. Everything above prices that configuration without running it. We now run it on a shipped runtime that stores the cache packed: llama.cpp build `b12063` (CUDA 12.4), whose `-ctk`/`-ctv` flags hold the key and value cache in a per-32-block form and which dispatches flash attention for a quantized V. The weights are the `Q8_0` GGUF of each model — the weight side of the recommendation, not a different one — and the cache is swept over fp16, q8_0 and q4_0 at four lengths.

The decisive cell is the 4B model at 32k. With an fp16 cache the runtime allocates 9,033 MiB on a device with 8,188: the configuration produced no token — an attempt ran for 20 minutes pinned at 7,840 MiB with the GPU at 100% utilization, and the measurement run was then cut at its 421 s cell timeout. With the cache at q8_0 the same length needs 6,873 MiB and completes 32k of prefill plus 128 decode steps in 67 s. *On this device, 32k context for a 4B model exists because the cache is stored packed* — not because any quantizer was chosen well.

The price is small, and it is paid in the right phase. Packing the cache costs prefill throughput and leaves decode alone: at a fixed length the three cache types give decode within 3% of one another (1.7B at 32k: 117.7, 115.0, 115.7; 4B at 16k: 53.4, 52.5, 52.7), while prefill falls by 6–10% (1.7B at 32k: 4,281 to 3,861 and 3,962; 4B at 16k: 2,514 to 2,391 and 2,425). That is the same qualitative split the simulated rows suggested, now measured on a packed implementation rather than on our dequantizing patch.

TABLE VIII
THE RECOMMENDED POLICY EXECUTED ON LLAMA.CPP `b12063`, ON DEVICE A (8188 MIB PHYSICAL, 6.9 GIB USABLE). "ALLOCATED" IS THE SUM OF THE RUNTIME'S OWN CACHE, COMPUTE AND MODEL BUFFERS, READ FROM ITS ALLOCATION LOG; PREFILL AND DECODE ARE TOKENS PER SECOND. NOTHING IN THIS TABLE IS MODELLED.

model	L	cache	alloc (MiB)	prefill	decode
1.7B	4096	fp16	2,508	8,617	106.0
		q8_0	2,298	8,044	104.9
		q4_0	2,186	8,167	104.9
	32768	fp16	5,672	4,281	117.7
		q8_0	3,992	3,861	115.0
		q4_0	3,096	3,962	115.7
4B	16384	fp16	6,713	2,514	53.4
		q8_0	5,633	2,391	52.5
		q4_0	5,057	2,425	52.7
	32768	fp16	**9,033**	does not complete	
		q8_0	6,873	1,634	52.6
		q4_0	5,721	1,657	52.6

Accuracy is where the two models separate, and the separation is large. At L=4096 on WikiText-2 (16 chunks) an 8-bit cache is free on both GPU devices — 15.6078 to 15.6257 on 1.7B, 11.8091 to 11.8099 on 4B — but 4 bits costs +26.15% on the smaller model (19.6896) and +0.99% on the larger (11.9257). Both runs log `flash_attn = enabled` with the same graph, so this is not a code path switching under us: one quantizer, one runtime, 26× difference in the cost of a cache bit between two models of the same family. Any default of the form "4 bits is fine for the cache" is a claim about one model that has been generalised to others.

A second device. The Orin Nano Super is the unified-memory endpoint of the same argument: 8 GB nominal, no separate VRAM, and the budget that Section III measured at $\rho = 0.419$. We build the same runtime tag on the device (sm_87, CUDA 12.6) and load the same GGUF. On this platform the runtime maps the weights instead of copying them into a device buffer, so the cache is what competes for the budget.

Table IX is a ladder: 8k at 16 bits (896 MiB), 16k at 8 bits (952), 32k at 4 bits (1,008). Each halving of the per-token cache buys exactly one doubling of attainable context, and the rung above each step is refused in all three attempts (1,792 and 1,904 MiB). What makes this a capacity result rather than an out-of-memory result is the measured peak: the largest working cell peaks at 5,235 of 7,620 MB, so the device is not full when it refuses a 1,792 MiB allocation. The runtime reports 4,482 MiB free at that moment, and a plain CUDA program can allocate 1,793, 238 and 952 MiB in the same order — so the refusal belongs to the runtime's allocation pattern, which maps the weights first, and not to the hardware. That is the paper's thesis in miniature: the budget a configuration gets is neither the number on the box nor the number the runtime reports.

TABLE IX

THE SAME POLICY ON A UNIFIED-MEMORY DEVICE: DEVICE B (8 GB, 25 W, DESKTOP RUNNING), LLAMA.CPP B12063, QWEN3-1.7B WITH Q8_0 WEIGHTS. "KV" IS THE CACHE THE RUNTIME ALLOCATES, NOT A MODEL BUFFER IT DOES NOT COPY; "REFUSED" MEANS THE ALLOCATION FAILED IN ALL THREE ATTEMPTS. RUN-TO-RUN SPREAD IS LARGE (PREFILL SD TO 26%, DECODE SD TO 24% AT 4K WITH AN 8-BIT CACHE); ROWS ARE SINGLE RUNS AND NO TIMING CLAIM RESTS ON THEM.

L	cache	KV (MiB)	prefill	decode
4096	fp16	448	1,209	24.3
	q8_0	238	989	24.6
	q4_0	126	1,139	22.9
8192	fp16	896	1,069	21.4
	q8_0	476	1,040	22.0
	q4_0	252	1,024	25.0
16384	fp16	1,792	refused	
	q8_0	952	854	24.3
	q4_0	504	876	22.8
32768	fp16	3,584	refused	
	q8_0	1,904	refused	
	q4_0	1,008	648	24.2

Accuracy reproduces across devices, and only the budget does not. The same quantizer on the same GGUF costs $+0.10\%$ at 8 bits (15.1822 to 15.1968) and $+25.06\%$ at 4 bits (18.9865) on the Orin, against $+0.11\%$ and $+26.15\%$ on the laptop. The policy that follows is therefore device-specific and computable: on this device q8_0 at 16k occupies the same budget as fp16 at 8k (952 against 896 MiB), doubles the context, and costs no measurable accuracy, which is a better trade than 4 bits at 32k — a third of the cache for a quarter more perplexity.

Four boundaries. The scheme is llama.cpp's per-32-block packing, not the grouped quantizer of [20], so the claim is that a shipped runtime stores the cache packed and the latency and footprint it achieves doing so; we do not claim that grouped-quantization kernel ran. Windows does not raise an out-of-memory error when a configuration exceeds dedicated memory, it pages, so the device peaks in Table VIII saturate near 7,800 MiB whether or not a configuration fits; the decisive evidence is the allocation log against the physical size, which is why we report allocated rather than peak. And these are two GPU devices plus a CPU device, differing in runtime and in weight format, which bounds the reading to the existence proof the capacity argument needs. A fourth caution runs the other way: instantiation is decided by ambient memory, so a single refusal is not evidence of a frontier — a 16k q8_0 attempt failed once and then succeeded three times, which is why Table IX reports three attempts per cell rather than one.

E. The cache is not the fragile side

Quantization practice spends its bits on weights and leaves the cache in 16 bits, on the implicit assumption that the cache is the part that breaks first. Table X measures both sides the same way — quantize every linear layer at a given width and record the perplexity cost, then quantize the cache at the same width and record the cost — and finds the opposite: at 4 bits the cache costs 5.35% and the weights cost 34.65%; at 3 bits the cache costs 25.2% while the weights collapse outright.

TABLE X

RELATIVE PERPLEXITY INCREASE WHEN ALL WEIGHTS, OR THE WHOLE KV CACHE, ARE QUANTIZED TO THE SAME WIDTH, ON TWO MODEL SIZES. CORPUS: 2,000 ARXIV ABSTRACTS, 48 WINDOWS OF 1,024 TOKENS; INTERVALS ARE 95% BOOTSTRAP CIS OVER WINDOWS. THE "TUNED" ROW STRENGTHENS THE WEIGHT SIDE AS FAR AS GROUP SIZE AND GRID SYMMETRY ALLOW AND IS THE HONEST WORST CASE FOR OUR OWN CLAIM. THE ORDERING — CACHE CHEAPER THAN WEIGHTS AT THE SAME WIDTH — REPRODUCES AT 1.7B, BUT THE MARGIN UNDER A TUNED QUANTIZER SHRINKS TO ABOUT ONE PERCENTAGE POINT ON BOTH. †THE WEIGHT ENTRY IS A GENUINE 8-BIT QUANTIZATION; THE CACHE ENTRY IS THE FP16 REFERENCE.

bits	weights	KV cache	ratio
Qwen3-0.6B			
8†	-0.08%	0.00%	—
4	$+34.65\%$ [33.2, 36.1]	$+5.35\%$ [4.9, 5.9]	$6.5\times$
4 (tuned)	$+6.97\%$ [6.4, 7.5]	$+5.35\%$ [4.9, 5.9]	$1.30\times$
3	collapse	$+25.23\%$	—
2	collapse	collapse	—
Qwen3-1.7B			
4	$+11.33\%$ [10.1, 12.5]	$+1.98\%$ [1.4, 2.5]	$5.7\times$
4 (tuned)	$+3.21\%$ [2.6, 3.8]	$+1.98\%$ [1.4, 2.5]	$1.62\times$

Both intervals exclude zero, so the ordering is not a small-sample accident.

We repeated the comparison on a second checkpoint, Qwen3-1.7B, because a single model is the obvious objection to any claim of this shape. The ordering reproduces and the ratio is close ($5.7\times$ against $6.5\times$), although both costs fall in absolute terms (11.33% and 1.98%): the smaller model is the more weight-sensitive one, which is consistent with weight quantization mattering less as models grow. The tuned margin also reproduces at about one percentage point, so the two models agree on both the direction and the size of the effect.

The tuned rows matter for how strongly this result can be stated. Our weight quantizer is round-to-nearest, while GPTQ and AWQ extract substantially more from the same width, so a reviewer is entitled to ask whether the ordering is an artefact of a weak quantizer. Strengthening the weight side as far as group size and grid symmetry allow brings its cost from 34.7% down to 6.97% — close to the cache's 5.35%, but not below it: the paired difference, cache minus weights on the same windows, is -1.5 percentage points with a 95% interval of $[-2.16, -0.84]$, which excludes zero but only just. We therefore claim the weaker, safer statement: *with quantizers of comparable strength the cache is not the fragile side*, so treating it as untouchable while compressing weights lacks a basis. The published GPTQ implementation, run at the same width below, does not close that margin either — it widens it — but we still do not claim that a cache bit is always cheaper. The un-tuned 34.7% figure is reported for completeness but should not be compared against the cache number: it measures round-to-nearest, not the state of the art.

Is the rate a property of the bit axis, or of one 4-bit point?: The columns above fix one operating point per side, which invites the objection that the rate is an artefact of comparing 4 bits against 4. We measured the joint grid instead

TABLE XI
THE SAME COMPARISON UNDER THREE QUANTIZER PAIRS, EACH STRONGER THAN THE LAST, WITH CALIBRATION WINDOWS HELD OUT FROM EVALUATION. THE CACHE IS CHEAPER PER BIT IN EVERY SYMMETRIC PAIR, BY BETWEEN 3.2× AND 14.3× (THE LAST COLUMN IS WEIGHTS OVER CACHE, SO A VALUE BELOW ONE MEANS A CACHE BIT IS THE DEARER ONE). THE RATE BELONGS TO THE PAIR, NOT TO THE BIT AXIS: THE SAME 4-BIT CACHE COSTS 1.98% UNDER KIVI ON 1.7B AND 26.15% UNDER LLAMA.CPP'S Q4_0 BLOCKS (TABLE VIII), AND WE DO NOT SEPARATE THE MECHANISMS. THE SINGLE ROW WHERE IT IS NOT — WEIGHTS AT GROUP 16 AGAINST AN UNGROUPED CACHE — IS AN ARTEFACT OF GIVING ONE SIDE FINER GRANULARITY THAN THE OTHER; WE REPORT IT BECAUSE WE PRODUCED IT OURSELVES AND IT IS THE NATURAL WAY TO GET THIS COMPARISON WRONG.

		rel. PPL (%)		weights/cache
weights	cache	0.6B	1.7B	0.6B / 1.7B
RTN g128	per-ch/tok	34.71	11.45	6.2 / 6.2
AWQ g128	KIVI R128	16.45	9.61	3.2 / 4.5
AWQ g16	KIVI+g16	3.27	2.75	4.2 / 14.3
AWQ g16	KIVI R128	3.27	2.75	**0.6 / 1.3**
asymmetric: weights grouped, cache not — not a valid comparison				

— weights in $\{2,3,4,5,6,8\}$ bits, cache in $\{2,3,4,5,6,8\}$, both quantizers in the same run, Qwen3-0.6B, six windows of 1,024 tokens, baseline PPL 44.54, rel. PPL $= 100(e^{\Delta_{\text{nats}}}-1)$. Two regimes appear. At the aggressive end the weight bit is the expensive one. With 5- and 6-bit cache cells measured, on a matched 4→6 interval a weight bit costs 9.4× a cache bit at K=4 and 15.5× at K=8 over 48 windows, with a four-fold scatter of at most 1.6 points; over six windows the same interval gives 4.8× and 10.2× (3.61 against 17.48 points per bit at K=4), so the sign and the order of magnitude are stable while the second digit is not. We name the two quantities once: R_{point} is the ratio of two same-width relative costs, and R_{slope} the ratio of per-bit slopes on a matched interval. Above 6 bits both curves saturate: the cache slope falls to 0.05–0.07 points per bit and the weight slope to 0.19–1.20, so the ratio stops separating them within the fold-to-fold scatter. The rate is therefore a property of the operating point, not a constant; the 9–13× headline is the aggressive-weight-width regime and is stated as such. Below 4 bits both sides collapse (PPL 2,109 at W3/K4 against 44.5 at 16) and no allocation matters.

The two axes rank differently in accuracy and in capacity: At 0.6B the weights take 531 MB at 4 bits and 752 MB at 8 against 7.44 GB usable on the laptop, while the 1k cache is 15–29 MB. With the measured ρ (Eq. 1) the attainable context is 38,410 tokens at W4/K4 against 32,885 at W4/K8: halving the cache width costs 14% of context, halving the weight width 3%. Accuracy and capacity therefore rank the two axes oppositely, and any single published configuration resolves that trade once, for one geometry.

Against the shipped recipe at a fixed budget: Four-bit weights with an fp16 cache — the shipped recipe — cost +36.67% perplexity at 4,096 tokens in 766 MB, while eight-bit weights with an eight-bit cache cost +1.49% in 869 MB. The shipped recipe fits in 12% fewer bytes and pays 35 points more perplexity, and the crossover in *bytes* falls between 4k and 8k, where an fp16 cache outgrows the faster arm. We could not extend it to 8k: our chunked evaluator disagreed with single-pass scoring, so long-context evidence rests on Table VII. The comparison also holds when the cache quantiser is not ours. KIVI's own quantiser at eight bits, under the same four-bit weights and the same windows, costs +36.65% in 649 MB: compressing only the cache at a fixed weight width moves perplexity by two hundredths of a point, while spending 13% more bytes on both sides at once reaches −0.15% (869 MB).

Table VII.

The same protocol on a model from a different family, SmolLM2-135M: at 4,096 tokens the shipped recipe costs +101.68% in 204 MB and eight-bit weights with an eight-bit cache cost +0.56% in 210 MB: 3% more bytes, 101 points less perplexity. Its full grid, weights $\{2,3,4,6,8\}$ against caches $\{2,3,4,8\}$, reproduces both regimes: at 4-bit weights a cache bit costs 10.1 points against 49.0 for a weight bit (4.9×), while between 6 and 8 bits the weight slope falls to 1.9 and the cache, at 1.7, is no longer the cheap side. At 3 B, where full-precision weights still fit the budget, the same grid is harsher on the weight side: 4-bit weights cost +149.56% where 8-bit cost +6.46%, so the aggressive weight width is dearer than at 0.6 B and the cache remains the cheaper side by more, not less. On the module at 32k the 3,584 MiB fp16 cache buffer cannot be allocated at all, and the weights do not change that: at 4-bit weights it still fails with 4.9 GB free, so the ceiling is one allocation rather than the weight stream. A 4-bit cache fits, and halving the weights raises decode from 26.5 to 32.2 tokens/s at unchanged prefill (546 against 556), and quartering them to a 2-bit k-quant (736 MiB) raises it again to 36.2: at a fixed cache width the weight stream, not the cache, sets the decode rate.

Does the ordering survive stronger quantizers on both sides?: The tuned row strengthens only the weights, so it answers whether our conclusion is a weak-weight-quantizer artefact, not whether a state-of-the-art cache compressor keeps the cache the cheaper side. We implemented both — activation-aware per-channel scaling in the spirit of AWQ with the per-layer exponent chosen on held-out windows, and a KIVI-style cache with a 128-token full-precision residual window and optional grouping.

Two things follow. The ordering is stable under symmetric strengthening: the cache is 3.2–14.3× cheaper per bit across three quantizer pairs and two models, and the ratio does not drift monotonically with strength — it depends on the pair, not on one side. The one reversal, weights at group-16 against an ungrouped cache, makes the cache look 1.6× *more* expensive on 0.6B; it is an artefact of unequal treatment that we produced ourselves.

Both sides from published implementations: Everything above uses at least one quantizer of ours, so the last check removes both: the weights are quantized by the published `gptqmodel` (v4.2.5, second-order error feedback, 4 bits, group 128) and the cache by KIVI's own quantizer (their `utils_quant`, per-channel keys with sequence grouping, per-token values with head-dimension grouping, and a 128-token residual window)

The result is not marginal. At 0.6B the GPTQ weights cost +11.39% against a cache cost of +1.27% — a 9.0× margin.

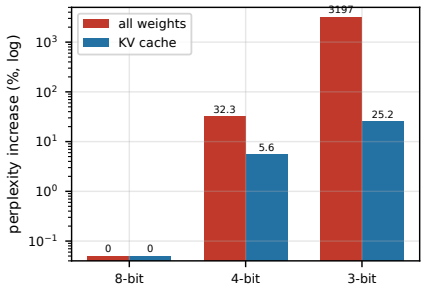


Fig. 3. Perplexity cost of quantizing all weights versus the whole cache. At 3 bits the weights collapse while the cache still costs only 25%; below that the comparison stops being meaningful.

At 1.7B the same pair gives $+3.20\%$ against $+0.24\%$, a $13.3\times$ margin. For comparison, our own KIVI-style adapter costs $+1.65\%$ and $+0.95\%$ on the two models, so their grouping is better than ours, which is the expected direction and the reason to use their code.

KIVI ships no Qwen3 model file, so the attention plumbing is ours in both cases, and we run their quantizer in simulation rather than through Triton kernels, unavailable on Windows: the comparison is of schemes at equal width, not kernels. The fp16 reference NLL agrees to five decimals across the two machines (3.850999 against 3.850961), which licenses comparing numbers measured on either.

Does the cache's cost accumulate with context?: The cache is quantized per token, and prior work reports error accumulation over autoregressive decoding, so one might expect its relative cost to grow. It does not: from 512 to 4,096 tokens with the token budget held fixed the cache cost moves from 6.71% to 6.85%, while the weight cost moves from 32.6% to 37.9% with no trend, so the ratio stays between 4.9 and 6.3.

The allocator built on these curves is free to choose any per-layer width and, at every accuracy budget we tried, leaves the weights at 8 bits and spends the budget on the cache (Table VI); a stronger cache quantizer would push the same way.

F. Negative results

Quantization has no effect on latency here (Section VI-B), so any claim that a bit-allocation scheme accelerates edge inference must rest on a different mechanism.

G. Threats to validity

Three confounds could make a measured number mean something other than it appears to, and we address each in the protocol rather than the prose.

The 4-bit path is bitsandbytes nf4, whose dequantization is not fused into the matmul; a kernel-fused quantizer could show a speedup that we do not, so the "no speedup" result is a statement about the current mainstream edge inference stack — the bitsandbytes/transformers path that an 8 GB device runs out of the box — rather than a universal claim about 4-bit arithmetic. We measure on exactly that path, so the scope of the claim and the scope of the deployment are the same. Background load moved a short-context prefill time by 10% between two consecutive runs in our own measurements, which is why compared configurations are interleaved and every ratio in Table V comes from paired runs. And the latency tables are computed on an architecture-exact random-weight model for the cases where no real checkpoint was available; we verified that this reproduces real-weight memory within 1% and naive-path prefill within 4% before relying on it, and every number in Tables IV and V is from real weights. Scope limitations — two model sizes and two corpora for the accuracy curves, one device actually run end to end — are separate and listed in Section VIII.

VII. Reproducibility

Every number in this paper is produced by a script in the artifact, and every table is generated from a measurement file rather than transcribed. A single command rebuilds the figures, the summary CSV, the allocator inputs, and the manuscript, and runs the checks below; it currently passes end to end.

- *Static manuscript checks* for unbalanced environments, table column counts, math delimiters, undefined references and missing bibliography entries, plus a *numeric audit* that extracts every factor and percentage claim and compares the headline ones against the result files. We added the audit after a revision changed a table without changing the abstract that quoted it; it found four such stale numbers.
- *A paired bootstrap* over evaluation windows, so the ordering is stated with an interval instead of a point, and *a model-free reproduction* of the attention-dispatch finding (Section VI-A) that needs no checkpoint.
- *Solver checks*: invariants plus a brute-force enumeration on small instances, which confirms the dynamic program is exact when the byte grid is fine enough rather than merely plausible.

We also record the execution stack with every measurement, including the package indices that were reachable from our network, because Section VI-A shows that the stack can change attainable context by $4\times$ — an effect larger than any allocation decision in this paper.

A task-level check where the width decides: Perplexity is an average; it cannot tell whether a model still finds something it was told 11,000 tokens ago, which is what a compressed cache threatens. We generated fifty prompts, one hidden passkey each, at 11k and 16k tokens, and ran five cache schemes (Table XII).

The first result is why we keep the 4-bit configurations: at that width the scheme does not matter for this capability, so the perplexity differences we measure are differences in fluency, not retrieval. The second corrects us. An earlier five-prompt run had suggested that KIVI's grouping and residual window rescued the 2-bit case; with fifty prompts per cell it does not — both 2-bit schemes recover 0/50 at both lengths, so for this capability there is a cliff at two bits that no scheme we tested crosses, and a short-context perplexity check does not reveal it. On the other family the same cliff appears: at 6,000 tokens,

TABLE XII
PASSKEY RETRIEVAL, QWEN3-0.6B, FIFTY GENERATED PROMPTS PER ROW AT 11K AND 16K TOKENS (THE TWO LENGTHS GIVE IDENTICAL COUNTS; WILSON 95% INTERVALS ARE $[0.93, 1.0]$ AND $[0.0, 0.07]$). "GROUPED" IS KIVI'S PER-CHANNEL KEYS, PER-TOKEN VALUES AND 128-TOKEN RESIDUAL WINDOW; "UNGROUPED" IS PLAIN ROUNDING. AT FOUR BITS THE SCHEMES DO NOT SEPARATE; AT TWO BITS BOTH FAIL.

cache	width	recovered
none (fp16)	16	50/50
ungrouped	4	50/50
grouped (KIVI)	4	50/50
grouped (KIVI)	2	**0/50**
ungrouped	2	**0/50**

TABLE XIII
THE STRENGTH-MATCHED COMPARISON REPEATED ON THE WIKITEXT-2 TEST SET, SAME WINDOW PROTOCOL, SAME CODE, SAME CALIBRATION DISCIPLINE. THE ORDERING HOLDS ON BOTH MODELS AND THE MARGIN IS *larger* THAN ON THE ABSTRACT CORPUS, SO THE CONCLUSION DOES NOT DEPEND ON THE CORPUS.

	Qwen3-0.6B	Qwen3-1.7B
weights, round-to-nearest	$+37.20\%$	$+28.49\%$
weights, AWQ-style (tuned)	$+17.52\%$	$+6.33\%$
cache, simple	$+5.25\%$	$+1.30\%$
cache, KIVI-style	$+5.34\%$	$+2.42\%$
cache, KIVI-style + grouping	$+\mathbf{1.61}\%$	$+\mathbf{0.82}\%$
matched ratio (weights / cache)	$10.9\times$	$7.7\times$

inside its $8{,}192$-token window, fp16 and both 4-bit schemes recover 30, 29 and 28 of 30 keys, and both 2-bit schemes none.

The ordering reproduces on a standard corpus: The results above use 2,000 arXiv abstracts, which invites the objection that the numbers are an artifact of the corpus. We therefore repeated the strength-matched comparison on the WikiText-2 test set, which became reachable through a mirror after the rest of the study was complete (Table XIII).

On WikiText-2 the matched weights cost 17.52% against 1.61% for the cache on 0.6B ($10.9\times$), and 6.33% against 0.82% on 1.7B ($7.7\times$). Both margins fall inside the range the abstract corpus produces, and the direction is identical, so the per-bit ordering is a property of the models and the quantizers rather than of the text. The deployment and passkey experiments remain on the abstract corpus so that they stay comparable with the measurements taken before WikiText-2 was available; what changed is that the central claim now has a standard-benchmark reproduction.

VIII. LIMITATIONS

Our accuracy curves are measured on Qwen3-0.6B and 1.7B, and the ordering claim is scoped to them: we could not add a 4B point on equal terms (a full-precision reference does not fit in 8 GB, and under the mismatched nf4/int8 setup the cache looks *more* expensive, $+9.46\%$ against $+6.66\%$), so we do not know where the ordering reverses. We also ran a real-workload check: 108 questions cut from the public LogHub log samples, answered under the shipped, our and full-precision configurations, gave 19.4%, 27.8% and 25.0% with overlapping intervals, so we do not use it as evidence of a task-level gain. The third-party check is split across devices (0.6B GPTQ on the module, 1.7B on the laptop, licensed by fp16 reference NLL agreeing to five decimals), and calibration size matters more than expected: moving from 2 to 16 texts made GPTQ *worse* ($11.39\% \to 13.0\%$ at 0.6B, $3.20\% \to 5.96\%$ at 1.7B) while the cache costs 1.27% and 0.24%, so the ordering survives either way but absolute weight costs are calibration-sensitive. Capacity beyond the measured range is analytic and the module figure is an upper bound: direct measurement puts 0.6B at $3{,}072$ tokens and aborts at $4{,}096$ where the model predicts 67k. The cache-compressed rows are in-attention fake-quantization, the packed execution is llama.cpp's per-32-block scheme rather than our kernel, and no published end-to-end allocator is released for this setting, so the comparison is against configurations, not against other allocators. At 2 bits neither scheme recovers an 11k-token passkey ($0/50$), so the task-level result holds only at the widths we measured; absolute perplexities are not comparable to published numbers on either corpus. Finally, the gains come from allocation and from repairing a dispatch failure, not from custom kernels: a fused kernel would change the constants and, as Section VI-A shows for the dispatch case, possibly the conclusions.

IX. CONCLUSION

The split between weight bits and cache bits is a decision, and on an 8 GB device the literature does not measure it. We replaced its three inputs with measurements: the objective is capacity, because 4-bit weights buy no time on the unfused kernels this device class runs; the exchange rate is a property of the operating point, 4.8–$15.5\times$ on a matched $4\to6$ interval and saturating above; and the budget is device-dependent through an unusable-memory fraction of 0.133 versus 0.419. What we do not establish is a win over a published allocation method at equal memory, an accuracy curve at 3–8B, or a second embedded module.

DATA AVAILABILITY

All measurement scripts, the vendored third-party quantizers, the files behind every table, and the figure-regeneration scripts are public at https://github.com/BRAD-cording/llm-8gb-bit-costs, pinned at commit `a6ee375` (MIT; figures CC BY 4.0). One command rebuilds the derived artifacts — figures, summary tables, allocator inputs and this manuscript — from those committed files; re-running the GPU measurements is a separate documented path.

REFERENCES

[1] S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, P. Cameron, M. Jaggi, D. Alistarh, T. Hoefler, and J. Hensman, "QuaRot: Outlier-free 4-bit inference in rotated LLMs," 2024, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2404.00456

[2] Z. Liu, C. Zhao, I. Fedorov, B. Soran, D. Choudhary, R. Krishnamoorthi, V. Chandra, Y. Tian, and T. Blankevoort, "SpinQuant: LLM quantization with learned rotations," 2024, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2405.16406

[3] H. Lin, H. Xu, Y. Wu, J. Cui, Y. Zhang, L. Mou, L. Song, Z. Sun, and Y. Wei, "DuQuant: Distributing outliers via dual transformation makes stronger quantized LLMs," 2024, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2406.01721

[4] Y. Sun, R. Liu, H. Bai, H. Bao, K. Zhao, Y. Li, J. Hu, X. Yu, L. Hou, C. Yuan, X. Jiang, W. Liu, and J. Yao, "Flatquant: Flatness matters for LLM quantization," 2024, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2410.09426

[5] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," 2022, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2210.17323

[6] E. Frantar, S. P. Singh, and D. Alistarh, "Optimal brain compression: A framework for accurate post-training quantization and pruning," 2022, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2208.11580

[7] Z. Dong, Z. Yao, Y. Cai, D. Arfeen, A. Gholami, M. W. Mahoney, and K. Keutzer, "HAWQ-V2: Hessian aware trace-weighted quantization of neural networks," 2019, neurIPS 2020. [Online]. Available: https://arxiv.org/abs/1911.03852

[8] Z. Zheng, X. Song, and C. Liu, "MixLLM: LLM quantization with global mixed-precision between output-features and highly-efficient system design," 2024, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2412.14590

[9] Z. Yao, H. Zhao, H. Wang, and X. Han, "GAMMA: Global bit allocation for mixed-precision models under arbitrary budgets," 2026, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2605.18475

[10] H. Zhang, Q. Zhang, Q. Cao, H. Fan, Y. Huang, K. Sha, and Y. Feng, "AlignQuant: Tile-aligned mixed-precision quantization for efficient LLM generation," 2026, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2610.07457

[11] K. Wang, Z. Liu, Y. Lin, J. Lin, and S. Han, "Haq: Hardware-aware automated quantization with mixed precision," 2018, arXiv preprint. [Online]. Available: https://arxiv.org/abs/1811.08886

[12] Y. Lin, H. Tang, S. Yang, Z. Zhang, G. Xiao, C. Gan, and S. Han, "Qserve: W4a8kv4 quantization and system co-design for efficient LLM serving," 2024, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2405.04532

[13] Y. Zhao, C.-Y. Lin, K. Zhu, Z. Ye, L. Chen, S. Zheng, L. Ceze, A. Krishnamurthy, T. Chen, and B. Kasikci, "Atom: Low-bit quantization for efficient and accurate LLM serving," 2023, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2310.19102

[14] J. Wei, S. Cao, T. Cao, L. Ma, L. Wang, Y. Zhang, and M. Yang, "T-mac: Cpu renaissance via table lookup for low-bit LLM deployment on edge," 2024, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2407.00088

[15] H. Guo, N. Guo, W. Wang, J. Otholt, C. Meinel, and H. Yang, "Apex4: Efficient pure W4A4 LLM inference via intra-sm compute rebalancing," 2026, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2606.08761

[16] H. Lu, Z. Chen, G. Fang, X. Ma, and X. Wang, "Mix-Quant: Quantized prefilling, precise decoding for agentic LLMs," 2026, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2605.20315

[17] S. Song, G. Lee, J. Lee, and J. Choi, "Align, then correct: Training-free two-stage low-rank compensation for extremely quantized large language models," 2026, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2610.08164

[18] Y. Park, H. Kim, and S. Choi, "Serq: Saliency-aware low-rank error reconstruction for LLM quantization," 2026, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2603.08185

[19] W. Liu, H. Meng, Y. Luo, Y. Zhao, P. Zhang, and X. Ma, "Micromix: Efficient mixed-precision quantization with microscaling formats for large language models," 2025, iCLR 2026. [Online]. Available: https://arxiv.org/abs/2508.02343

[20] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, "KIVI: A tuning-free asymmetric 2bit quantization for KV cache," 2024, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2402.02750

[21] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, "KVQuant: Towards 10 million context length LLM inference with KV cache quantization," 2024, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2401.18079

[22] T. Zhang, Z. Zeng, H. Peng, H. Zhuang, and C. Chen, "MixKVQ: Query-aware mixed-precision KV cache quantization for long-context reasoning," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. San Diego, California, United States: Association for Computational Linguistics, 2026, pp. 7189–7204. [Online]. Available: https://aclanthology.org/2026.acl-long.326/

[23] S. Gokhale, D. Das, R. Patwari, A. Sirasao, and E. Delaye, "KV Pareto: Systems-level optimization of KV cache and model compression for long context inference," in *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 5: Industry Track)*. Rabat, Morocco: Association for Computational Linguistics, 2026, pp. 119–131. [Online]. Available: https://aclanthology.org/2026.eacl-industry.9/

[24] H. Duanmu, Z. Yuan, X. Li, J. Duan, X. Zhang, and D. Lin, "Skvq: Sliding-window key and value cache quantization for large language models," 2024, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2405.06219

[25] Z. Su, Z. Chen, W. Shen, H. Wei, L. Li, H. Yu, and K. Yuan, "Rotatekv: Accurate and robust 2-bit KV cache quantization for LLMs via outlier-aware adaptive rotations," 2025, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2501.16383

[26] Y. He, L. Zhang, W. Wu, J. Liu, H. Zhou, and B. Zhuang, "ZipCache: Accurate and efficient KV cache quantization with salient token identification," 2024, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2405.14256

[27] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," 2023, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2309.06180

[28] M. Chen, Y. Liu, J. Wang, Y. Bin, W. Shao, and P. Luo, "PrefixQuant: Eliminating outliers by prefixed tokens for large language models quantization," 2024, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2410.05265

[29] X. Hu, Y. Cheng, D. Yang, Z. Xu, Z. Yuan, J. Yu, C. Xu, Z. Jiang, and S. Zhou, "OstQuant: Refining large language model quantization with orthogonal and scaling transformations for better distribution fitting," 2025, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2501.13987

[30] D. Haldar, I. Patra, A. Yousuf, and C. S. Seelamantula, "SpectrumQuant: Spectrum alignment for LLM quantization," 2026, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2610.20303

[31] Y. Li, R. Yin, D. Lee, S. Xiao, and P. Panda, "GPTAQ: Efficient finetuning-free quantization for asymmetric calibration," 2025, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2504.02692

[32] Z. Yao, Z. Dong, Z. Zheng, A. Gholami, J. Yu, E. Tan, L. Wang, Q. Huang, Y. Wang, M. W. Mahoney, and K. Keutzer, "HAWQV3: Dyadic neural network quantization," 2020, ICML 2021. [Online]. Available: https://arxiv.org/abs/2011.10680

[33] A. Panferov, M. Kleinegger, S. Priyadarshi, T. Blankevoort, and D. Alistarh, "Disaggregated quantization: Specializing LLM prefill and decode," 2026, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2609.26333

[34] Y. Zhou, Z. Wu, R. Jiang, D. Ren, Y. Han, W. Zhao, D. Cai, and W. Li, "MultiAlloc: Budget-aware precision allocation for KV cache quantization," 2026, arXiv:2610.18642.

[35] M. Kim, H. R. Sung, S. Ko, H. Cho, J. Lee, J.-Y. Kim, and J. Park, "Oaken: Fast and efficient LLM serving with online-offline hybrid KV cache quantization," in *Proc. 52nd Annual International Symposium on Computer Architecture (ISCA)*, 2025, doi:10.1145/3695053.3731019.

[36] H. Liu, L. Zhai, J. Wang, Z. Fang, J. Chen, and J. Huang, "SparKV: Overhead-aware KV cache loading for efficient on-device LLM inference," 2026, arXiv:2604.21231; under review at IEEE Internet of Things Journal.

[37] S. P. H. Boroujeni, N. Mehrabi, P. Woods, G. Hillesheim, and A. Razi, "Don't waste bits! adaptive KV-cache quantization for lightweight on-device LLMs," 2026, arXiv:2604.04722; in Proc. IEEE/CVF CVPR 2026.

[38] C. Slothouber, "The kv4 trade-off is workload-dependent: A depth- and workload-resolved study of 4-bit KV-cache quantization on a 4 GB Turing GPU," 2026, sSRN preprint, doi:10.2139/ssrn.6941538.

[39] H. Zhang, J. Guo, L. Benini, and Y. Li, "RDKV: Rate-distortion bit allocation for joint eviction and quantization of the KV cache," 2026, arXiv:2605.08317.

[40] S. Bhandari, L. Gretta, K. Balasubramanian, and S. Kasiviswanathan, "Read what matters: Query-adaptive quantization for KV caches," 2026, arXiv:2610.11245.

[41] D. Lee and H. Song, "Q-Palette: Fractional-bit quantizers toward optimal bit allocation for efficient LLM deployment," 2025, arXiv:2509.20214.